COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010 / Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	October 13, 2026	**DATE SUBMITTED:**	October 15, 2026
NAME:	Aguila, Daniel C. Torres, Carl Andrei D.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. HOA 13.1 Parallel Algorithms and Multithreading

1. PROCEDURE OUTPUT

What procedures did you perform?

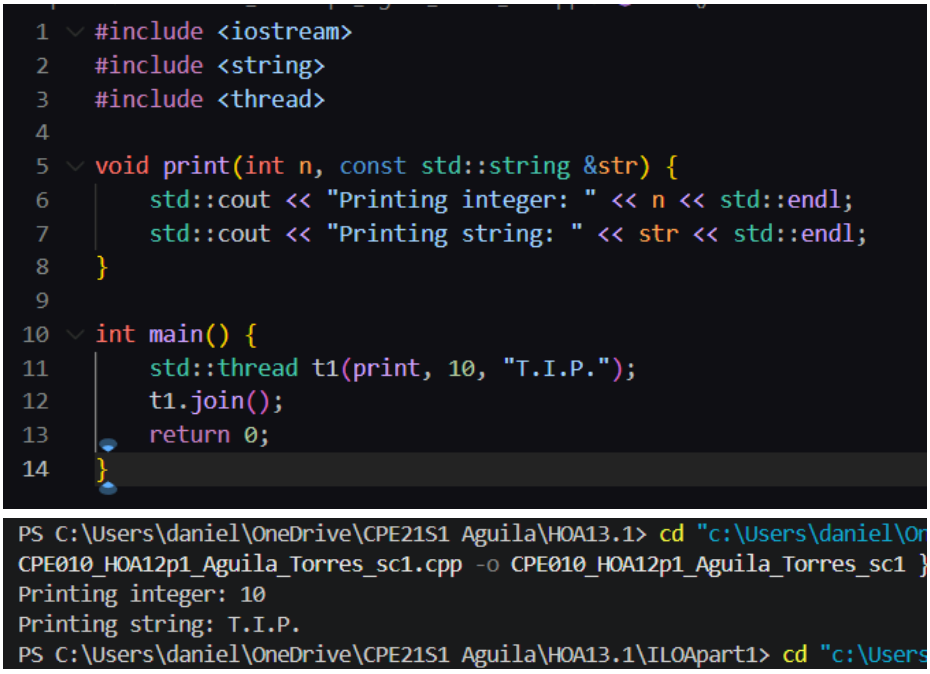
ILO A & ILO B:

PART 1: SIMPLE ONE-THREADED EXAMPLE

Since all threads must be given a function to complete at their creation, we first must declare a function for it to be given. We'll name this function print and will design it to take int and string arguments when called. When executed, this code will simply report the data values passed in.

In the next section, we'll initialize a thread and have it execute the above function. To do this, we'll have the main function, the default executor present in all C++ applications, initialize the thread for the print function.

After that, we use another handy multithreading command, join(), pausing the main function's thread until the specified thread, in this case t1, has finished its task. Without join() here, the main thread would finish its task before t1 would complete print, resulting in an error.

Screenshot	
Analysis	**Aguila** I've seen that one thread named `t1` is created and it executes `print(10,"T.I.P.")`. The use of `join()` causes `main()` to wait for the completion of `t1`that make sure that the output is displayed before the end of the program execution. Without `join()`, `main()` would execute first and terminate the program. Also, the `main()` does not have anything else to do other than waiting for `t1`so no speed up in this case. **Torres** I've analyzed that one thread that named "t1" is being implemented and it executes a output 'print(10, "T.I.P"). The use of the "join()" causes the main driver to wait for the completion of the function callable "t1" that makes sure the output is displayed successfully before the end of the program to execute.

```cpp
#include <iostream>
#include <string>
#include <thread>

void print(int n, const std::string &str) {
    std::cout << "Printing integer: " << n << std::endl;
    std::cout << "Printing string: " << str << std::endl;
}

int main() {
    std::thread t1(print, 10, "T.I.P.");
    t1.join();
    return 0;
}
```

```
PS C:\Users\daniel\OneDrive\CPE21S1 Aguila\HOA13.1> cd "c:\Users\daniel\Or
CPE010_HOA12p1_Aguila_Torres_sc1.cpp -o CPE010_HOA12p1_Aguila_Torres_sc1 }
Printing integer: 10
Printing string: T.I.P.
PS C:\Users\daniel\OneDrive\CPE21S1 Aguila\HOA13.1\ILOApart1> cd "c:\Users
```

PART 2: MULTI-THREADED EXAMPLE

While the outcome of the single thread example above could easily be replicated without using multithreaded code, we can truly see concurrency's benefits when we attempt to complete print multiple times with different sets of data. Without multithreading, this would be done by simply having the main thread repeat print one at a time until completion.

To do this with concurrency in mind, we instead use a for loop to initialize multiple threads, pass them the print function and arguments, which they then complete concurrently. This multithreading option would be faster one using only the main thread as more of the total CPU is being used.

Runtime difference between multithreading and non-multithreading solutions increasing as more print executions are needed.

Let's see what a many-thread version of the above code would look like:

Screenshot	
Analysis	**Torres:** As we can see in the graph, I've implemented the callable function which prints the number of the string was being executed and the string itself from the list ("T.I.P", "Competent", "Computer", "Engineers") which has a sample output of (0 : Competent). Also, I've implemented the for loop which has a push_back function to insert a new

```cpp
#include <iostream>
#include <thread>
#include <vector>
#include <string>

void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;
}

int main() {
    std::vector<std::string> s = {
        "T.I.P.",
        "Competent",
        "Computer",
        "Engineers"
    };

    std::vector<std::thread> threads;

    for (int i = 0; i < s.size(); i++) {
        threads.push_back(std::thread(print, i, s[i]));
    }

    for (auto &th : threads) {
        th.join();
    }
    return 0;
}
```

```
PS C:\Users\daniel\OneDrive\CPE21S1 Aguila\HOA13.1\ILOB> cd "c:\Users\da
CPE010_HOA13p1_Aguila_Torres_sc2.cpp -o CPE010_HOA13p1_Aguila_Torres_sc2
0 : T.I.P.2 : Computer

1 : Competent
3 : Engineers
PS C:\Users\daniel\OneDrive\CPE21S1 Aguila\HOA13.1\ILOB>
```

	element until the very end of the list of elements. Lastly, I've implemented the multithreading which the th.join() for loop to manage the execution of the function callables which has a different output every execution. **Aguila** Based on the code shown above, the `<thread>` header basically allows multithreading in the programs as it supplies the class `std::thread`. Every `std::thread(print, i, s[i])` call in `main` produces an instance of the class which begins executing `print` right away with the specified parameters while `push_back` puts it in the `threads` vector. It is because the objects of `std::thread` cannot be copied that the move semantics of the header enables such an operation. It is also in the header where the `join()` member function is defined and used by the second loop to wait until all threads complete their execution.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Part A: Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions. Use of supplementary materials to support answers must be cited as reference.

Questions:

1. Write a definition of multithreading and its advantages/disadvantages

Carl's Answer	Daniel's Answer
The **multithreading programming concept refers to an execution model which allows a process to execute multiple threads, which run concurrently while sharing the same memory and processing**. The **main advantages of this approach are that it provides better responsiveness in applications by not freezing the graphical interface when a background thread is performing some operations, and provides a way to share resources easier and more efficiently than its independent processes**. **Multithreading also allows to take advantage of using multiple CPU cores by running independent threads on separate cores**. However, the **main disadvantages of multithreading are it adds significant complexity to the code which makes debugging harder and errors such as its**	Multithreading refers to **running multiple threads in one application whereby all the threads run different functions while sharing the same memory**. The pros include utilization of the CPU cores resulting in completion of major tasks quickly, and responsiveness of the application because one thread runs in the background. The cons include data race and deadlocks, unpredictability in the sequence of output, difficult debugging process, and creation of threads is time-consuming, thus it is a waste of resources for minor tasks.

data race conditions or deadlocks possible. Another disadvantage is that **threads require more system resources and can lead to extra memory overhead due to context switching between threads, which can reduce its overall performance**.	

2. Rationalize the use of multithreading by providing at least 3 use-cases.

Carl's Answer	Daniel's Answer
Multithreading is really essential in application development because it prevents the single-threading executions and maximizes the hardware efficiency across multi-core architectures. In **Graphical User Interfaces (GUI), offloading the heavy disk or network operations to background threads that keeps the UI fully interactive and efficient**. Moreover, **Web Servers utilize multithreading to serve multiple concurrent client connections simultaneously without waiting for a single request to finish**. Lastly, the **modern media tools and game engines uses this approach to parallelize some intensive tasks like real-time video rendering, physics simulations, and audio processing**.	An example is a **web server which spawns a separate thread for each client request to ensure that one user does not stop others and several users can be serviced simultaneously.** The second example is **software and computer games which utilize separate threads to do the actual job in the background while the main thread maintains the smooth operation of the graphical interface, thus avoiding program freezing**. The last example is **heavy computations such as sorting of arrays, image processing or performing mathematical operations on massive data sets which can be divided into smaller parts and performed by several different cores at once**, similar to what we have done with the merge sort algorithm in Part C where two halves were sorted simultaneously by separate threads. These examples show appropriate usage of threads since tasks are either independent from each other or massive enough to justify thread spawning cost.

3. Differentiate between parallelism and concurrency.

Carl's Answer	Daniel's Answer
Concurrency refers to the software capability to handle multiple tasks by overlapping their execution time, often achieved by rapid context switching on a single CPU core. On the other hand, **Parallelism requires multi-core hardware to physically execute multiple instructions at the same time which makes the execution efficient across multi-core architectures**. Essentially, **Concurrency is about managing and structuring many tasks at once**, while **Parallelism is about**	Concurrency, **it talks about several processes running in parallel in time with possible data exchange between them, as well as sharing the memory.** It also may be achieved even on one core as the CPU merely performs task switching. This situation may be compared to the two queues served by one espresso machine in turns. Parallelism **involves several tasks running exactly at the same moment and working on similar but different data on separate cores**. In this case, we have two queues

doing many tasks simultaneously.	and two espresso machines serving both lines at the same moment. Therefore, the key point is that concurrency is about managing several tasks, while parallelism is about processing them simultaneously with a need for several cores. Another difference is that there is a higher possibility of race condition in case of concurrency due to data sharing in contrast to the parallelism which should be independent.

Part B: Create C++ Code and show a solution that satisfies the given requirements below.
Create a global variable of type integer.
Create an add function that will take an integer parameter and add that value to the global variable.
Use multi-threading techniques to create 3 threads; individually pass the add function to the threads.
Display the value of global variable at different combinations of using the join() per thread. Such that:

o Display
o T1.join()
o Display
o T2.join()
o Display
o T3.join()

Provide an analysis based on the outputs of the multi-threading exercise.

```cpp
#include <iostream>
#include <thread>
#include <vector>
#include <string>
#include <chrono>


int globalValue = 0;


void add(int value) {

    std::this_thread::sleep_for(std::chrono::milliseconds(value * 10));

    globalValue += value;
}


void display(const std::string &label) {
    std::cout << label << " : globalValue = " << globalValue << std::endl;
}

```

```cpp

void display(const std::string &label) {
    std::cout << label << " : globalValue = " << globalValue << std::endl;
}

int main() {
    std::vector<int> values = {10, 20, 30};

    std::vector<std::thread> threads;

    for (int i = 0; i < values.size(); i++) {
        threads.push_back(std::thread(add, values[i]));
    }

    display("Display (before any join)");

    threads[0].join();
    display("Display (after T1.join)");

    threads[1].join();
    display("Display (after T2.join)");

    threads[2].join();
    display("Display (after T3.join)");

    return 0;
}
```

Code + Output Console

Analysis	**Torres:** I've analyzed that the program creates a three threads that each of the callables add with a different value (10, 20, 30) to update the global variable which is the globalValue functions. The first display, which was taken before any join() function and it shows 0 because the main thread keeps on running while the other threads are still on delay. After the T1.join() function, the value is 10. After the T2.join() function, it becomes 30. After the T3.join() function, it reaches a final value of 60. I've analyzed that the join() function makes the main thread wait for the given thread to finish, so the displayed value only includes the work of the threads that have already completed. **Aguila:** We created three threads to increment by 10, 20 and 30 to a global variable, then printed its value prior to any `join` and after every `join()`. We always get 0, 10, 30, then 60 as the output. This is due to the `sleep_for` in the `add` function that causes T1, T2 and T3 to sleep for 100, 200 and 300 ms respectively and thereby complete one by one. Prior to any `join` the value will be 0 since all the threads are still sleeping. After every `join` main waits for each thread and consequently the value increases till 60. This looks predictable due to the sleep mechanism. But the threads still share the variable without using a mutex.

Part C: Use multi-threading with one of the algorithms previously developed in the course; provide an analysis of the result.

```cpp
#include <algorithm>
#include <chrono>
#include <iostream>
#include <random>
#include <thread>
#include <vector>

void merge(std::vector<int> &a, int l, int m, int r) {
    std::vector<int> tmp(r - l + 1);
    int i = l, j = m + 1, k = 0;
    while (i <= m && j <= r) tmp[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];
    while (i <= m) tmp[k++] = a[i++];
    while (j <= r) tmp[k++] = a[j++];
    std::copy(tmp.begin(), tmp.end(), a.begin() + l);
}

void mergeSort(std::vector<int> &a, int l, int r) {
    if (l >= r) return;
    int m = (l + r) / 2;
    mergeSort(a, l, m);
    mergeSort(a, m + 1, r);
    merge(a, l, m, r);
}
```

```cpp
void mergeSortThreaded(std::vector<int> &a, int l, int r, int depth) {
    if (l >= r) return;
    if (depth <= 0) { mergeSort(a, l, r); return; }
    int m = (l + r) / 2;
    std::thread left(mergeSortThreaded, std::ref(a), l, m, depth - 1);
    mergeSortThreaded(a, m + 1, r, depth - 1);
    left.join();
    merge(a, l, m, r);
}

int main() {
    const int N = 2000000;
    std::mt19937 rng(42);
    std::vector<int> base(N);
    for (auto &x : base) x = rng() % 1000000;

    std::vector<int> a = base;
    auto t0 = std::chrono::high_resolution_clock::now();
    mergeSort(a, 0, N - 1);
    auto t1 = std::chrono::high_resolution_clock::now();
    std::cout << "Single-thread: "
              << std::chrono::duration_cast<std::chrono::milliseconds>(t1 - t0).count()
              << " ms, sorted=" << std::is_sorted(a.begin(), a.end()) << std::endl;

    for (int d : {1, 2, 3}) {
        std::vector<int> b = base;
        auto s = std::chrono::high_resolution_clock::now();
        mergeSortThreaded(b, 0, N - 1, d);
        auto e = std::chrono::high_resolution_clock::now();
        std::cout << "Threaded depth " << d << " (" << (1 << d) << " threads): "
                  << std::chrono::duration_cast<std::chrono::milliseconds>(e - s).count()
                  << " ms, sorted=" << std::is_sorted(b.begin(), b.end()) << std::endl;
    }
    return 0;
```

```
PS C:\Users\daniel\OneDrive\CPE21S1 Aguila\HOA13.1> cd "c:\Users\
multi-threading } ; if ($?) { .\multi-threading }
Single-thread: 1361 ms, sorted=1
Threaded depth 1 (2 threads): 702 ms, sorted=1
Threaded depth 2 (4 threads): 428 ms, sorted=1
Threaded depth 3 (8 threads): 271 ms, sorted=1
PS C:\Users\daniel\OneDrive\CPE21S1 Aguila\HOA13.1\SUPP\pC\pB> cd
ing.cpp -o multi-threading } ; if ($?) { .\multi-threading }
Single-thread: 1443 ms, sorted=1
Threaded depth 1 (2 threads): 660 ms, sorted=1
Threaded depth 2 (4 threads): 390 ms, sorted=1
Threaded depth 3 (8 threads): 356 ms, sorted=1
PS C:\Users\daniel\OneDrive\CPE21S1 Aguila\HOA13.1\SUPP\pC\pB>
```

Code + Output Console

Analysis	**Aguila**: For Part C, I **created a threaded version of merge sort in which the left half is sorted in a new `std::thread` while the current thread sorts the right half, and `left.join()` is used before `merge()`, so that both halves will be sorted by the time of joining. I measured the algorithm performance on 2,000,000 randomly generated numbers with depth 1, 2, and 3 (2, 4, and 8 threads) and compared it with the same algorithm without threading. The times for single-threaded algorithm was 1361 ms, and the times for multithreaded algorithms were 702 ms, 428 ms, and 271 ms, and in all cases `sorted=1` was printed out, so the result is still correct. There is no race condition in this implementation because each thread acts only with its own half of the vector, and the only shared operation is `merge` which occurs after `join`. [If faster: The multithreaded algorithm was faster because two halves are independent and there is no need to synchronize them, but there is only approximately 2x speedup because the last merge occurs on one thread.] [If not faster: This is what I have learnt that having more threads does not necessarily imply greater speed due to the amount of work per thread.** Torres: In this part, we **have implemented a merge sort using multithreading. The current thread sorts the right half while the left half is being sorted by a new std::thread**. **Using left.join() before merge() makes sure that both halves are sorted before merging. Benchmarking this implementation on 2,000,000 random numbers, shows that the single-threaded version took 1,361 ms to sort, while the multithreaded took 702 ms at depth 1 (two threads), 428 ms at depth 2 (four threads) and 271 ms at depth 3 (eight threads)**. In **each run the sorted=1 was printed to indicate that there are no data race conditions in the implementation**. There are **no data race conditions because the threads only work on separate parts of the vector and the only shared resource is merge() which always runs after left.join(), therefore, it cannot have race conditions**. Overall, the **multithreaded algorithm was much faster than the single-threaded one, because the sub-arrays do not need to share any resources, but it is not ideal due to the merge stage, which is single-threaded**.

Conclusion/Learnings/Analysis	**Aguila:** In this activity, I've learned that concurrency is when tasks overlap and parallelism is when they really run at the same time on different cores, and that we can use std::thread in C++ to run functions in separate threads, with join() making main wait for them to finish. In the procedure, the single thread just printed the values, while the multithreaded one printed the lines in a different order sometimes because the threads don't run in a fixed order. In the supplementary activity, Part B showed that join() is the safe point to read

	the global variable, and the values went 0, 10, 30, then 60 only because of the sleep_for since the threads still share the variable without a mutex and could cause a race condition. In Part C, the threaded merge sort gave the same sorted result as the normal one, but it was faster on our PC / it wasn't much faster since more threads added overhead so more threads doesn't always mean faster. Also, I think we've done good in this activity because we understood the basics and got the programs working but we still need to practice using mutex and atomic to avoid race conditions and measuring performance properly. **Torres:** In performing this Hands-On Activity, I've **discovered the difference between concurrency, a situation where two or more tasks are executed at the same time, and parallelism, an activity where tasks are run on different cores in a computer's central processing unit (CPU)**. Using **the C++ programming language and the std::thread library**, I **discovered how to execute two or more functions concurrently and use the join() function to make sure that the main thread of the program waits until all other threads have exited before proceeding to the next one**. In the main procedure, the **single thread version of the code printed numbers in a sequential manner while the other version printed the numbers in different orders depending on which thread printed them first**. In the supplementary activities, **Part B revealed that the join() function acts as a synchronization point for reading shared global variables as the code which had been made to sleep for a period of time before accessing the shared variable using the std::this_thread::sleep_for() function did not prevent a race condition from occurring**. On the other hand, **Part C indicated that the multithreaded merge sort was correct and faster than the single thread version but not by an order of magnitude due to the overhead involved with the synchronization process**. Altogether, **our team was able to implement concurrent programs correctly**. However, I **need to practice further by learning how to use std::mutex, implement atomic operations, and analyze performance.**

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **N/A**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.